Lab 03

Creating and Publishing an API through the Publisher

WSO2 API Manager 4.7.0 · Practical exercise

Before You Begin

URLs in this guide use placeholders. Replace them with the values of the API Manager you are using:

- **<APIM_HOST>** – host name or IP address of the API Manager server (localhost when API Manager runs on your own laptop).
- **<HTTPS_PORT>** – API Manager HTTPS port: 9443 plus any port offset (Publisher, Developer Portal, Admin Portal, Management Console, token endpoint).
- **<GW_HTTPS_PORT> / <GW_HTTP_PORT>** – API Gateway ports: 8243 (HTTPS) and 8280 (HTTP) plus any port offset.
- **<APIM_HOME>** – the folder where API Manager is extracted.

The servers use a self-signed certificate by default, so the browser shows a security warning the first time: choose Advanced and continue.

Training Objective

Learn how to create an API, add API documentation, and publish it to the Developer Portal and deploy to gateway using the Publisher.

Business Scenario

After setting up the API-M, the API is created, published and deployed through the API Publisher in order to make it discoverable and subscribable from the store (Developer Portal).

PizzaShack Limited is providing a store from which consumers can subscribe to their API. This works as a secondary business function for PizzaShack and attracts many developers to the PizzaShack website. The API will be comprehensively documented for ease of use.

High-Level Steps

- Add the PizzaShack API to the Publisher
- Add documentation
- Deploy to gateway
- Publish the API

Part 1 – Add the PizzaShack API to the Publisher

Now that API Manager is set up and the users exist, create the API that the PizzaShack application needs.

1. Open the Publisher:

```
https://<APIM_HOST>:<HTTPS_PORT>/publisher
```

CC BY 4.0

Example:

```
https://localhost:9443/publisher
```

2. Log in as the user with the creator role you defined in the previous lab (**creator**).
3. Click **Create API** (or go to **APIs** in the left navigation and click **Create API**).
4. Under **REST API**, select **Start From Scratch**.
5. Provide the API details as per the table below and click **Create**. (The creator user cannot use **Create & Publish**, because publishing needs the publisher role.)

Field	Value	Description
Name	`PizzaShack`	Name of the API, used to identify it. It must be unique: if PizzaShack already exists on a shared server, add your initials (for example PizzaShack-JD).
Display Name	`PizzaShack`	Name of API as you want it to appear in the API store (Developer Portal).
Context	`/pizzashack`	URI context path used by API consumers. It must also be unique (for example /pizzashack-jd on a shared server).
Version	`1.0.0`	API version (in the form of version.major.minor).
Endpoint	The URL in the box below	The sample back end that ships with API Manager. Keep localhost here: the gateway calls the back end on the same server. With no port offset, <HTTPS_PORT> is 9443. The endpoint is used for both production and sandbox.

Endpoint:

```
https://localhost:<HTTPS_PORT>/am/sample/pizzashack/v1/api/
```

Example:

```
https://localhost:9443/am/sample/pizzashack/v1/api/
```

6. Under **Portal Configurations**, select **Basic info**, provide the following information and click **Save**.

Field	Value	Description
Publisher Access Control	All	The ability to protect an API to be managed only by users with specific roles.
Developer Portal Visibility	Public	Whether this API is visible to all or restricted to certain roles in the Developer Portal.
Thumbnail Image	Upload PizzaShack-logo.png from the lab files (Day 1 folder). Click the thumbnail area to upload it.	Icon to be displayed in the Developer Portal (can be jpeg, tiff, png format).
Description	Enter the text in the box below this table.	High level description of API functionality.
Tags	`pizza, order, pizza-menu`	One or more tags. Tags are used to group/search for APIs (press Enter after each tag).

CC BY 4.0

API Categories	No API Categories defined by default. Hence leave this field blank. The categories should be defined by an Admin user through the Admin Portal.	API categories allow API providers to categorize APIs that have similar attributes. When a categorized API gets published to the Developer Portal, its categories appear as clickable links to the API consumers. The API consumers can use the available API categories to quickly jump to a category of interest.
GitHub URL	You can leave this field blank	This GitHub URL will be available in the API overview page in the Developer Portal.
Slack URL	You can leave this field blank	This Slack Channel URL will be available in the API overview page in the Developer Portal.
Mark the API as third party	No	Indicates if an API is a third party API. You can use third party APIs to expose an externally published API through API Manager.
Make this the default version	No	The default version option allows you to mark one API, from a group of API versions, as the default one, so that it can be invoked without specifying the version number in the URL.

Description:

```
PizzaShackAPI: Allows to manage pizza orders (create, update, retrieve orders).
```

7. Under **API Configurations**, select **Runtime**, configure the following and click **Save**.

Field	Value	Description
Transport Level Security	HTTP and HTTPS (default). Leave Mutual SSL off.	APIs can be exposed in HTTP and/or HTTPS transport: the transport protocol on which the API is exposed. Both HTTP and HTTPS transports are selected by default. If you want to limit API availability to only one transport (e.g., HTTPS), un-check the other transport. In mutual SSL the server validates the identity of the client so that both parties trust each other. If the Mutual SSL option is selected you can upload the certificates as necessary.
Application Level Security	OAuth2 (default). Optionally also select Basic and Api Key.	APIs published on WSO2 API Gateway can be secured by OAuth 2.0. This is the default security available for all APIs. Basic authentication is a simple HTTP authentication scheme in which the request will contain an authorization header with a valid base64 encoded username and password. API Manager uses a self-contained JSON Web Token (JWT) as the API key. You can enable all these authentication mechanisms at the same time if required.
Key Manager Configuration	Allow all	Resident Key Manager is the default key manager available. You can plug in multiple key managers if required. The Allow selected option gives you the capability to allow token generation from only a selected set of key managers.
CORS Configuration	Disabled	Enable CORS for the API.
Schema Validation	Disabled	Enables the request and response validation against the OpenAPI definition if selected.

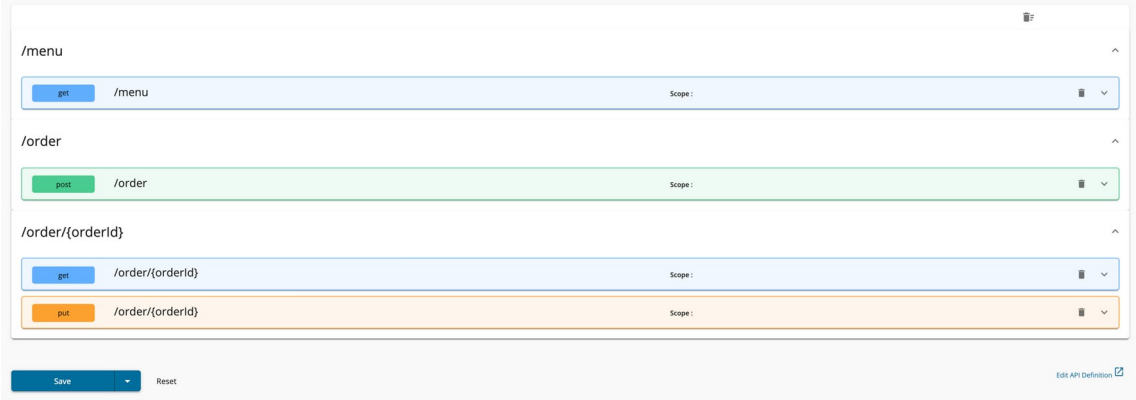
CC BY 4.0

Response Caching	Disabled	Response caching is used to enable caching of response messages per API. Caching protects the backend systems from being exhausted due to serving the same response (for the same request) multiple times. If you select the enable option, specify the cache timeout value (in seconds) within which the system tries to retrieve responses from the cache without going to the backend.
Backend Throughput	Unlimited	Limits the total number of calls the API Manager is allowed to make to the backend. While the other throttling levels define the quota the API invoker gets, they do not ensure that the backend is protected from overuse. Hard throttling limits the quota the backend can handle.

8. Under **API Configurations**, select **Resources**. **Start From Scratch** adds default resources with the URI pattern `/*` for GET, PUT, POST, DELETE and PATCH: delete all of them, add the four resources below, and click **Save**.

   For each resource, select the HTTP verb, type the URI pattern and click **Add** (the + button).

Resource URI Pattern	HTTP Verb
`/menu`	GET
`/order`	POST
`/order/{orderid}`	GET
`/order/{orderid}`	PUT



9. Select **Local Scopes** and click **Create Scope**. Add a scope with the information below. In this use case, ordering a pizza needs an OAuth2 scope named `order_pizza`, which only users with the admin or Internal/subscriber role can get. The PizzaShack web application used in the Invoking the API lab asks for exactly this scope name, so keep it as `order_pizza`.

   Scopes enable fine-grained access control to API resources based on user roles. You define scopes to an API's resources. When a user invokes the API, their OAuth2 bearer token cannot grant access to any API resource beyond its associated scopes.

Field	Description
Scope Name	A unique key for identifying the scope. Typically, it is prefixed by part of the API's name for uniqueness, but is not necessarily reader-friendly.
Roles	The user role(s) that are allowed to obtain a token against this scope. E.g., manager, employee. This field is optional.

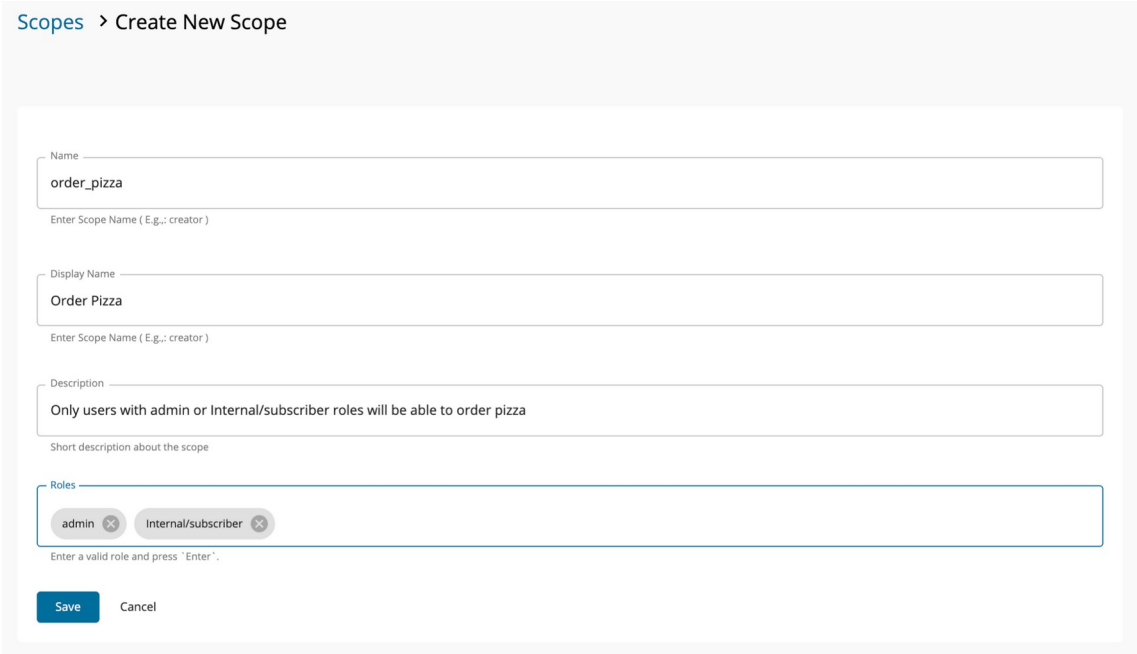
CC BY 4.0

To invoke a resource protected by a scope, the access token must include that scope. You can request it from the token endpoint, or select it when you generate a token in the Developer Portal.

Enter these values:

Field	Value
Name	`order_pizza`
Display Name	`Order Pizza`
Description	`Only users with admin role and internal/subscriber role can order`
Roles	`admin, Internal/subscriber`

(Press Enter after each role.)

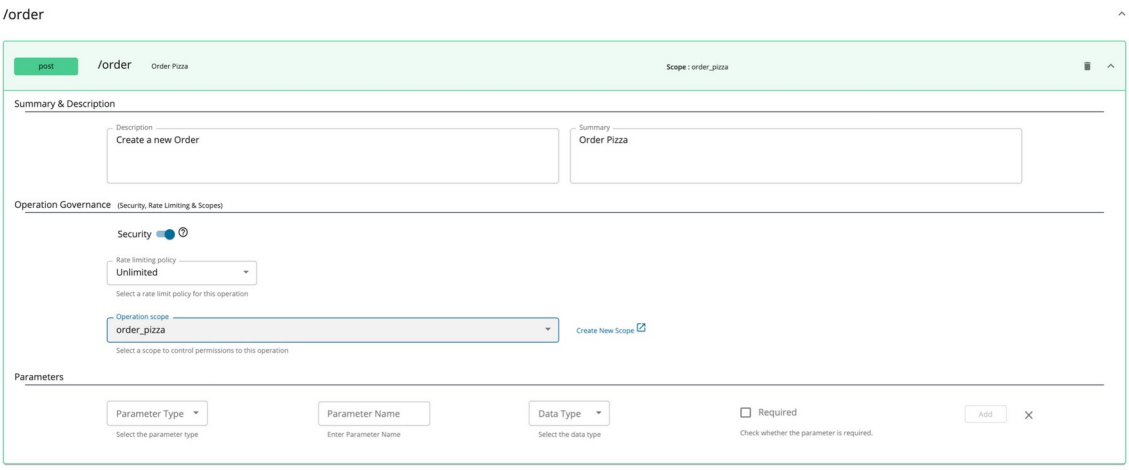


Since the scope is defined now, assign that scope to the appropriate API resources.

10. Select **Resources** again, expand the `POST /order` resource, add the following information and click **Save**.

Field	Value
Summary	`Order Pizza`
Description	`Create a new Order`
Operation Scope	Select order_pizza from the drop-down list

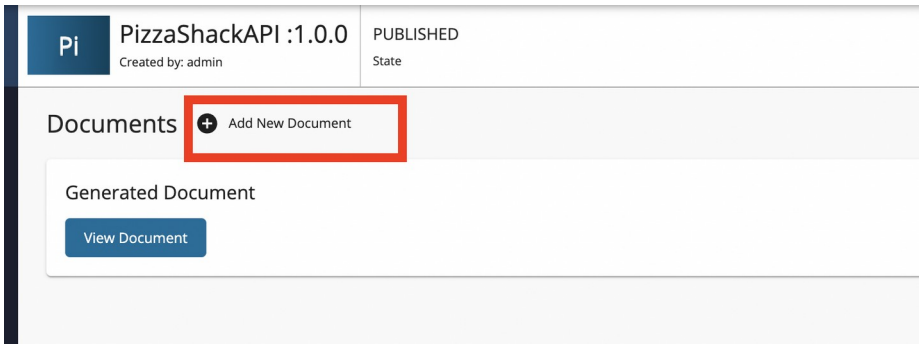
CC BY 4.0



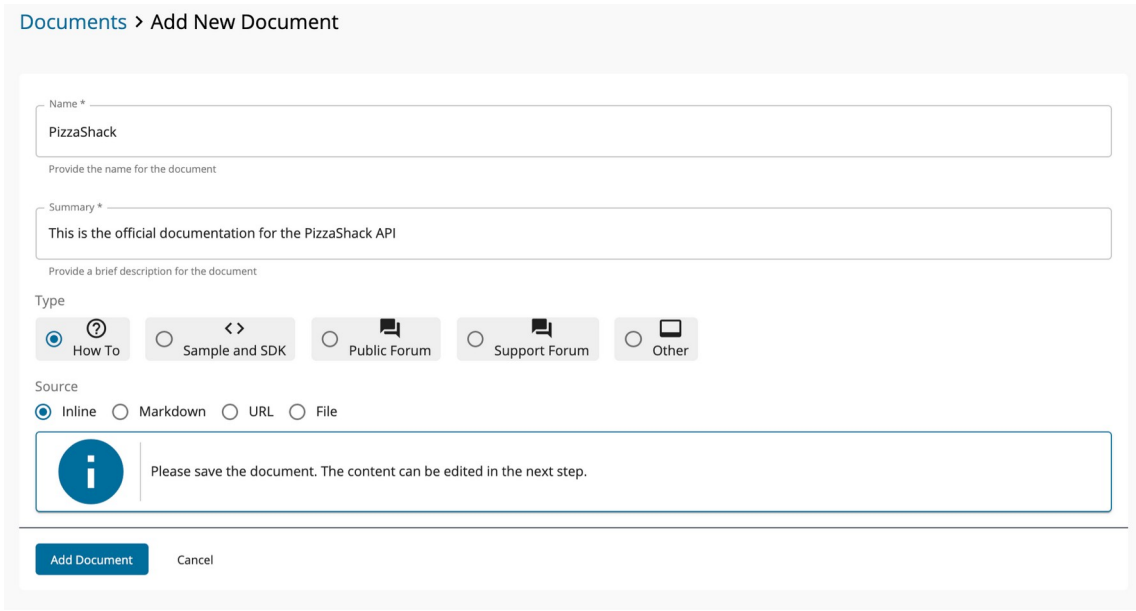
Part 2 – Add Documentation

11. Under **Portal Configurations**, select **Documents**.

12. Click **Add New Document**, enter the following details and click **Add Document**.



Field	Value
Name	`PizzaShack`
Summary	`This is the official documentation for the PizzaShack API`
Type	How To
Source	Inline



CC BY 4.0

Several document types are available:

- How To
- Samples and SDK
- Public Forum
- Support Forum
- Other

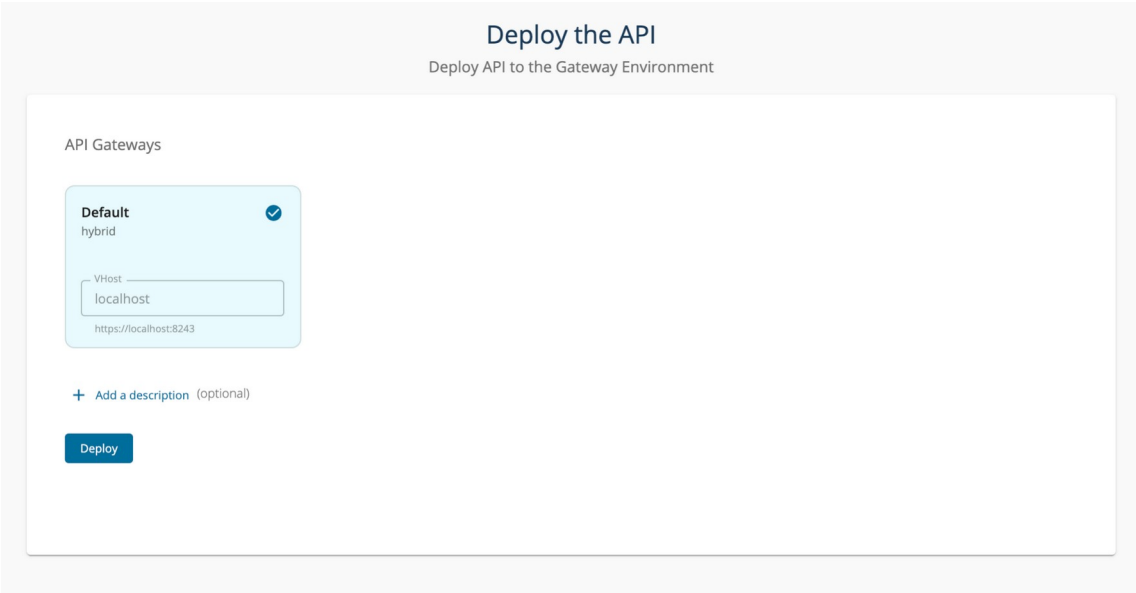
13. After the document is added, click **Edit Content** to write its content in the embedded editor, then click **Save**.

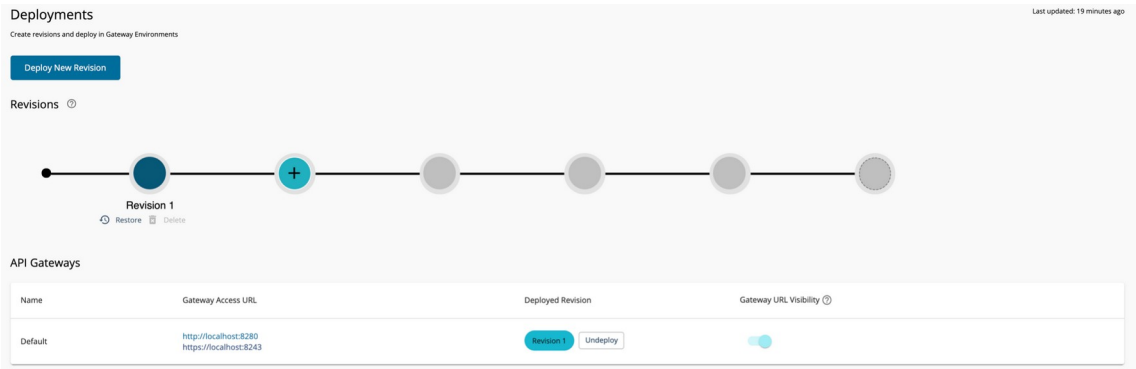
Part 3 – Deploy the API

Deploying makes the API available for invocation through a gateway. To invoke an API, it must be **DEPLOYED** on a gateway environment and **PUBLISHED** on the Developer Portal.

Each deployment uses a revision (a snapshot) of the API. Deploy the API while you are still logged in as **creator**, before it is published: once an API is published, only a user with the publisher role can create new revisions.

14. Under **Deploy**, select **Deployments**. Select the **Default** gateway (VHost `localhost`) and click **Deploy**. This creates Revision 1 and deploys it to the gateway.





Part 4 – Publish the API

The API is now ready to be published. This has to be done by a user with the publisher role.

CC BY 4.0

15. Log out as **creator** and log in as **publisher**.

16. Open the **PizzaShack** API. A **Lifecycle** page is now available in the left menu (under **Deploy**) for managing the API lifecycle.

17. Click **Publish**.



The API is now published and visible to consumers in the Developer Portal. The lifecycle history is shown at the bottom of the Lifecycle page.

Developer Portal:

```
https://<APIM_HOST>:<HTTPS_PORT>/devportal
```

Example:

```
https://localhost:9443/devportal
```

Expected Outcome

The PizzaShack API has been created by the creator user, documented, deployed to the gateway, and published by the publisher user. It is visible in the Developer Portal, and its POST /order resource is protected by the order_pizza scope.

CC BY 4.0